

Introduction: This program simulates a hybrid CPU scheduling algorithm using round-robin with Priority Scheduling. The program reads process data from a file, runs the simulation based on user-defined time quantum, and calculates performance metrics such as average waiting time, turnaround time, and CPU utilization.

CPU scheduling

CPU Scheduling is a crucial component in operating systems, where the CPU allocates time to various processes in an organized manner. Effective CPU scheduling maximizes CPU utilization and ensures efficient, fair distribution of CPU time among processes. Key metrics used to evaluate CPU scheduling performance include:

CPU Utilization: The percentage of time the CPU is actively working on processes.

Throughput: The number of processes completed per unit time.

Turnaround Time: The total time taken for a process to complete, from arrival to completion.

Waiting Time: The time a process spends waiting in the ready queue.

Response Time: The time from process arrival until its first response.

Scheduling Goals:

Maximize CPU Utilization and Throughput: Keep the CPU as busy as possible.

Minimize Turnaround and Waiting Times: Reduce delays for processes.

Fairness: Ensure processes are treated fairly, preventing starvation.

Round Robin

Round Robin (RR) is a CPU scheduling algorithm where each process is assigned a fixed time quantum. Processes are executed in a cyclic order, and if a process isn't completed within its time quantum, it's placed back at the end of the queue.

Key Properties of Round Robin:

Time Quantum: Each process receives a set amount of CPU time. The process is moved to the back of the queue if not completed.

Context Switching: The CPU switches between processes after each time quantum, which involves some overhead.

Fairness and Starvation Prevention: RR treats all processes equally, reducing the chance of starvation.

Impact of Time Quantum

Small Quantum: More context switching, higher overhead, but potentially better response time.

Large Quantum: Fewer context switches, but can lead to increased waiting time for processes.

How to run:

First, you need a Java IDE.

Second, make sure you have a CSV with the necessary information. (Example will be given)

Third, in the terminal, you would need to run "java Main"

Fourth, enter Quantum Time and the CSV file name.

Fifth, run it five times with different Quantum Times. Repeat steps 3-5 until you get 5 different results.

```
processes.csv
1  pid,arrive,burst,priority
2  1,0,5,2
3  2,1,7,1
4  3,2,3,3
5  4,3,2,4
6  
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE
PS C:\Users\alany\Desktop\OS\OP HW> java Main
Enter Quantum Time: 1
Enter File Name: processes.csv
Average Turnaround Time: 11.50
Average Waiting Time: 8.00
Average Response Time: 5.50
CPU Utilization: 100.00%
Throughput: 0.24 processes/unit time
Context Switches: 17
PS C:\Users\alany\Desktop\OS\OP HW> 
```

```
● PS C:\Users\alany\Desktop\OS\OP HW> java Main
Enter Quantum Time: 2
Enter File Name: processes.csv
Average Turnaround Time: 11.75
Average Waiting Time: 8.50
Average Response Time: 5.75
CPU Utilization: 100.00%
Throughput: 0.24 processes/unit time
Context Switches: 10
○ PS C:\Users\alany\Desktop\OS\OP HW> █
```

```
PS C:\Users\alany\Desktop\OS\OP HW> java Main
● Enter Quantum Time: 3
Enter File Name: processes.csv
Average Turnaround Time: 12.00
Average Waiting Time: 9.25
Average Response Time: 6.00
CPU Utilization: 100.00%
Throughput: 0.24 processes/unit time
Context Switches: 7
○ PS C:\Users\alany\Desktop\OS\OP HW> █
```

```
PS C:\Users\alany\Desktop\OS\OP HW> java Main
● Enter Quantum Time: 4
Enter File Name: processes.csv
Average Turnaround Time: 12.25
Average Waiting Time: 9.50
Average Response Time: 6.25
CPU Utilization: 100.00%
Throughput: 0.24 processes/unit time
Context Switches: 6
```

```
• Enter Quantum Time: 5
Enter File Name: processes.csv
Average Turnaround Time: 10.75
Average Waiting Time: 8.00
Average Response Time: 6.50
CPU Utilization: 100.00%
Throughput: 0.24 processes/unit time
Context Switches: 5
PS C:\Users\alany\Desktop\OS\OP HW>
```

Analysis

The size of the time quantum in Round Robin scheduling significantly impacts system performance. A small time quantum (e.g., 1) offers advantages such as lower average waiting time and faster response times, especially for interactive processes. However, this comes at the cost of higher context switching, which increases CPU overhead and reduces overall CPU utilization as more time is spent switching between processes rather than executing them. On the other hand, a large time quantum (e.g., 5) reduces the number of context switches, leading to higher CPU utilization and lower CPU overhead. This could be deduced from the screenshots above showing 5 context switches at 5 compared to 17 context switches at time 1. However, it results in higher average waiting times and longer response times, making the system less responsive to interactive tasks.

CPU utilization tends to improve with larger time quanta due to fewer context switches, but excessively large time quanta can lead to inefficiencies when processes have short burst times. Throughput generally increases with larger time quanta, as the system spends less time switching contexts and executing processes more. Priority scheduling further enhances responsiveness for critical processes without heavily penalizing lower-priority tasks. This approach balances system efficiency and user satisfaction. Selecting an optimal time quantum involves a trade-off between responsiveness and CPU utilization while integrating priority scheduling adds complexity but provides greater control over process management.

Extra Credit Analysis: It enhances the traditional round-robin scheduler by integrating priority-based scheduling, enabling higher-priority processes to be scheduled preferentially while maintaining the cyclic execution order within the same priority level. Processes are grouped by priority, with higher-priority groups scheduled before lower-priority ones, and round-robin scheduling applied within each group. This approach ensures a balanced allocation

of CPU time while respecting process priority. The scheduling algorithm manages ready queues either by using multiple queues for distinct priority levels or a single priority queue with priority-based ordering. The `runPriorityRoundRobin` method in my java file simulates this algorithm, taking a list of processes and a time quantum as inputs, and outputs a comprehensive set of performance metrics, effectively combining fairness and responsiveness in process management.

Conclusion

The Round Robin CPU Scheduling Simulator effectively demonstrates the principles and performance implications of the Round Robin algorithm. The simulator offers valuable insights into how scheduling parameters affect system performance metrics by enabling users to experiment with various time quantum values. Additionally, the Priority Round Robin version enhances the simulator's capabilities.